

AI-Based Network Pathfinding and Attack Simulation System

Comprehensive Professional Project Report

Submitted By	Muhammad Umer - FA24-BCS-112 Shifa Ullah - FA24-BCS-146 Hamza Khalid - FA24-BCS-091
Section	BCS-4B
Subject	Artificial Intelligence
Instructor	Maam Zeenat Zulfikar
Department	BS Computer Science
University	COMSATS University Islamabad

Abstract

This report presents a complete AI-based network pathfinding and attack simulation system implemented in Python. The project models a cybersecurity-inspired environment as a weighted graph and compares multiple Artificial Intelligence search strategies, including Breadth-First Search, Depth-First Search, Uniform Cost Search, A* Search, Hill Climbing, and Minimax with Alpha-Beta pruning. The system evaluates traversal paths from an attacker node to a target node while measuring path cost, expansion count, execution time, and algorithmic behavior.

The final implementation combines graph theory, heuristic search, interactive visualization, performance benchmarking, and adversarial decision modeling. NetworkX manages graph modeling, Matplotlib generates dynamic visual outputs, Pandas produces comparative result matrices, and ipywidgets supports interactive algorithm selection in Google Colab. The report also includes generated screenshots, network topology figures, UML-style diagrams, flowcharts, performance charts, dataset simulation, source code appendix, and viva preparation material.

The main finding is that A* Search provides the best overall balance between cost efficiency and heuristic intelligence, while UCS provides optimal cost guarantees at higher computational effort. DFS and Hill Climbing are faster but less reliable, and Minimax provides strategic reasoning but with greater recursive overhead.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives and Scope
4. Literature Review
5. System Requirements
6. Network Graph Formulation
7. System Architecture
8. UML Design
9. Algorithm Implementations
10. Visualization Engine
11. Generated Screenshots and Outputs
12. Dataset Simulation
13. Experimental Results
14. Performance Evaluation
15. Complexity Analysis
16. Security and Ethical Analysis
17. Limitations
18. Future Enhancements
19. Viva Questions and Answers
20. Appendix A: Source Code
21. Appendix B: LaTeX/IEEE Notes
22. References

1. Introduction

Modern cybersecurity environments contain multiple interconnected systems, public-facing services, internal resources, authentication layers, databases, and critical targets. Understanding how an attacker may move through such an environment is useful for learning security design and intelligent decision-making. This project simulates that movement through AI search algorithms over a weighted graph model.

Modern cybersecurity environments contain multiple interconnected systems, public-facing services, internal resources, authentication layers, databases, and critical targets. Understanding how an attacker may move through such an environment is useful for learning security design and intelligent decision-making. This project simulates that movement through AI search algorithms over a weighted graph model.

2. Problem Statement

The core problem is to determine how different AI search algorithms behave when they are asked to move from a source node named Attacker to a destination node named Target. A basic path may be easy to find, but an efficient path requires considering traversal cost, heuristic estimates, security resistance, and search behavior.

This core problem is to determine how different AI search algorithms behave when they are asked to move from a source node named Attacker to a destination node named Target. A basic path may be easy to find, but an efficient path requires considering traversal cost, heuristic estimates, security resistance, and search behavior.

3. Objectives and Scope

The objective is to implement and compare classical search algorithms under identical graph conditions. The scope is educational simulation, not real hacking. It models attack-path reasoning without performing exploitation, malware execution, traffic interception, or unauthorized access.

This objective is to implement and compare classical search algorithms under identical graph conditions. The scope is educational simulation, not real hacking. It models attack-path reasoning without performing exploitation, malware execution, traffic interception, or unauthorized access.

- Model a secured network as a weighted graph.
- Implement BFS, DFS, UCS, A*, Hill Climbing, and Minimax.
- Compare path length, traversal cost, expanded nodes, and execution time.
- Generate graph visualizations and performance charts.
- Prepare professional documentation suitable for academic evaluation.

4. Literature Review

Graph search is a foundational AI topic used in robotics, GPS navigation, game engines, puzzle solving, and network optimization. BFS and DFS are uninformed approaches. UCS uses actual cost. A* combines real cost with heuristic estimates. Hill Climbing uses greedy local improvement. Minimax supports adversarial decision-making, and Alpha-Beta pruning reduces search overhead.

Graph search is a foundational AI topic used in robotics, GPS navigation, game engines, puzzle solving, and network optimization. BFS and DFS are uninformed approaches. UCS uses actual cost. A* combines real cost with heuristic estimates. Hill Climbing uses greedy local improvement. Minimax supports adversarial decision-making, and

Alpha-Beta pruning reduces search overhead.

5. System Requirements

The system requires Python, NetworkX, Matplotlib, Pandas, Heap Queue utilities, Google Colab, and ipywidgets. Functional requirements include graph creation, algorithm execution, cost calculation, visualization, comparison reporting, and UI-based algorithm selection. Non-functional requirements include readability, modularity, reproducibility, and fast execution for small to medium graphs.

This system requires Python, NetworkX, Matplotlib, Pandas, Heap Queue utilities, Google Colab, and ipywidgets. Functional requirements include graph creation, algorithm execution, cost calculation, visualization, comparison reporting, and UI-based algorithm selection. Non-functional requirements include readability, modularity, reproducibility, and fast execution for small to medium graphs.

6. Network Graph Formulation

The graph is represented as $G = (V, E)$, where V is the set of network nodes and E is the set of weighted communication or traversal edges. Every edge weight represents resistance, access difficulty, or traversal cost. The attacker begins at the Attacker node and attempts to reach the Target node.

Node	Heuristic	Meaning
Attacker	10	Entry point
Firewall	8	Security barrier
Honeypot	3	Decoy trap
DMZ	7	Demilitarized zone
Web	6	Public web service
Mail	6	Mail server
Internal	5	Internal network
Auth	3	Authentication service
DB	2	Database server
Target	0	Final objective

The heuristic values are used by A* and Hill Climbing. A lower heuristic indicates that the node is estimated to be closer to the target or strategically more useful for reaching it.

7. Network Topology Figure

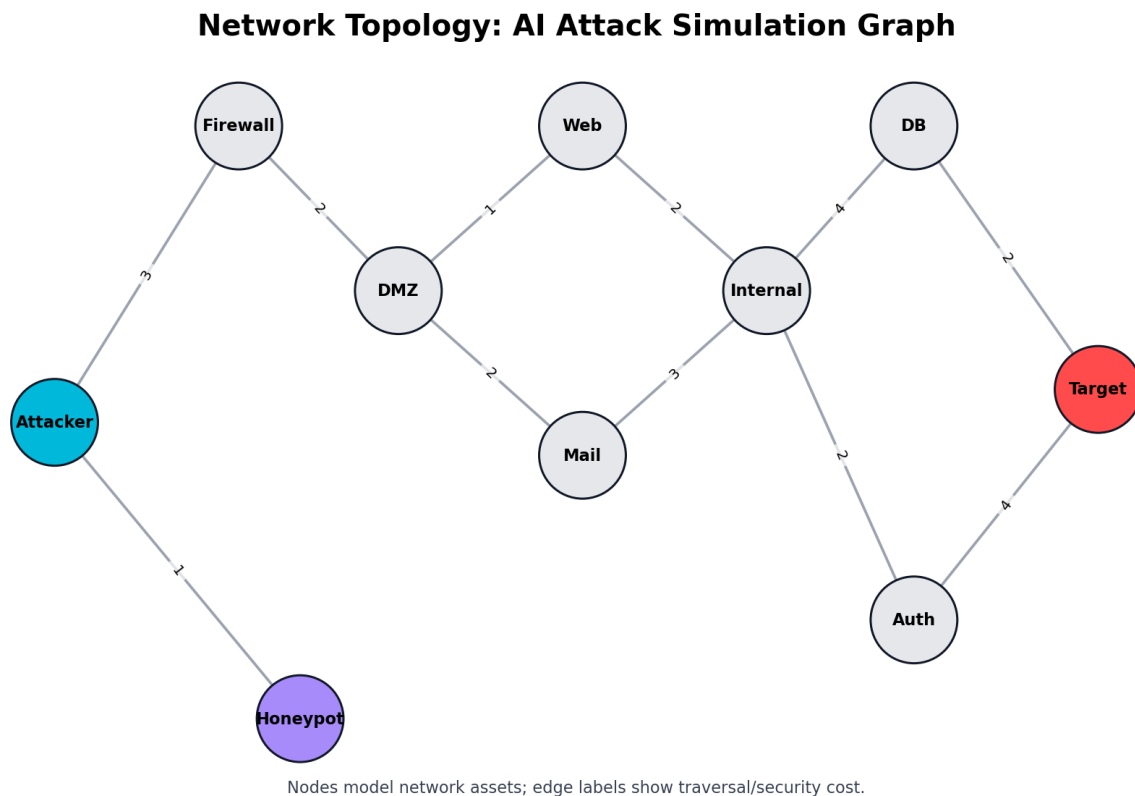


Figure 1. Weighted network topology generated from the project graph. Yellow/red highlighting is reserved for algorithm-specific paths in later figures.

This topology is the foundation of the simulation. It includes an attacker entry point, a firewall, a honeypot, a DMZ, public services, internal systems, authentication resources, a database, and a target system. The edge weights define traversal resistance.

8. System Architecture

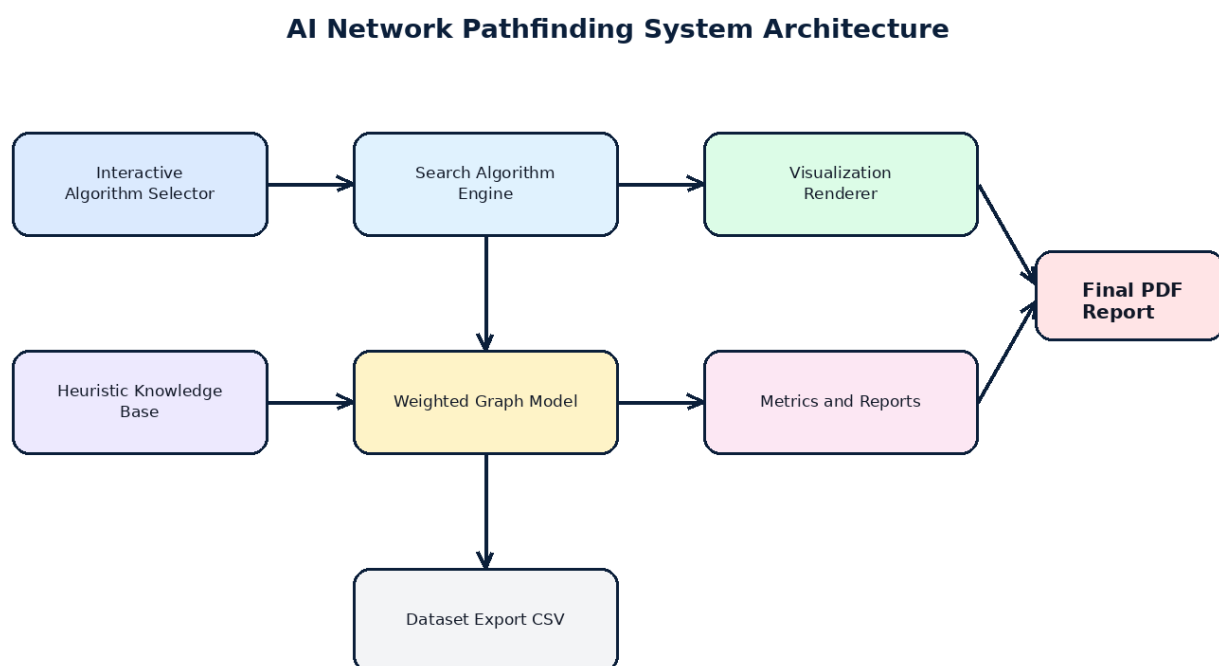


Figure 2. Auto-generated architecture image showing the modular design of the project.

The architecture separates the user interface, graph model, heuristic knowledge base, algorithm engine, visualization layer, metrics module, and report outputs. This separation makes the project easier to extend and debug.

9. UML Design

UML Class-Style Design

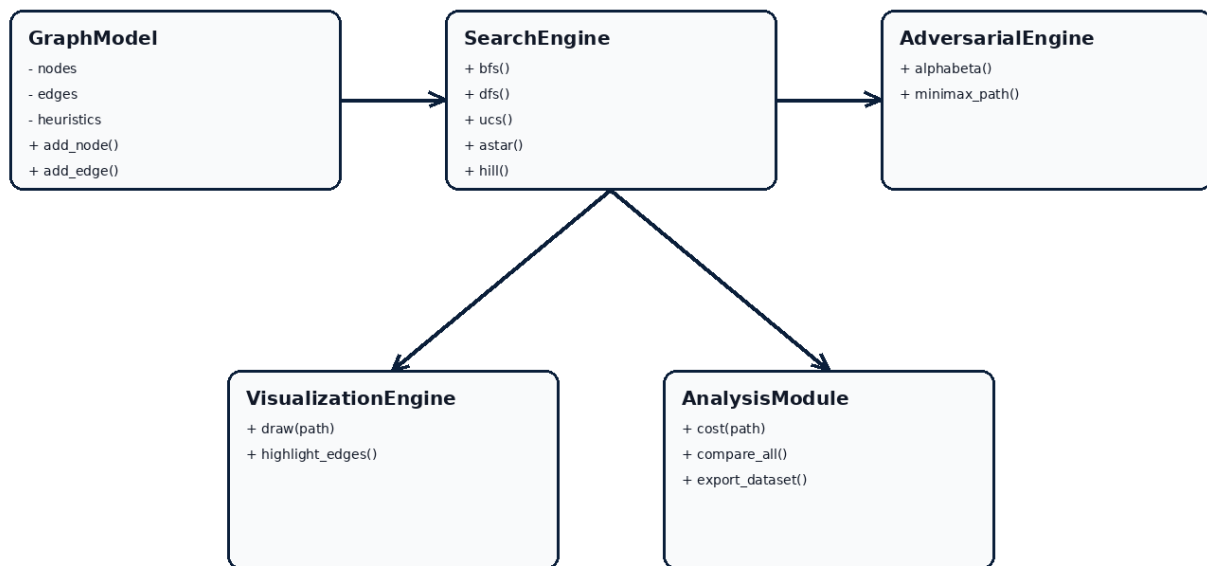


Figure 3. UML-style design showing project classes/modules and major functions.

The UML design shows how graph data, algorithm functions, visualization, and metrics interact. Although the implementation is functional in notebook style, this design explains how it could be converted into a clean object-oriented application.

10. Execution Flowchart

Execution Workflow Flowchart

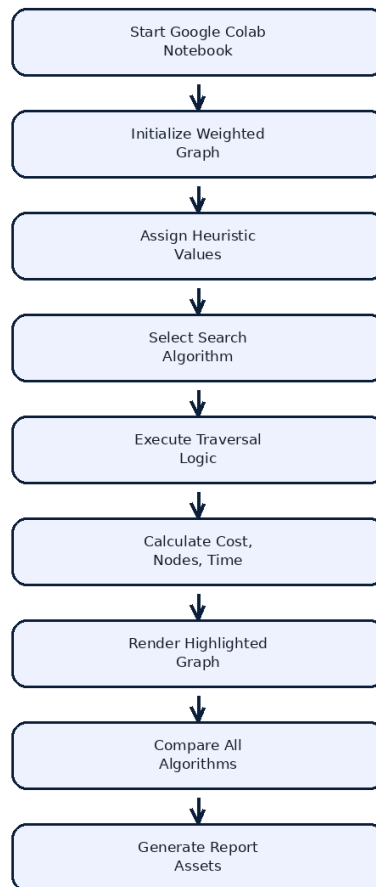


Figure 4. Flowchart showing the execution pipeline from graph initialization to report generation.

The execution starts with graph initialization, then heuristic assignment, algorithm selection, traversal execution, metric calculation, visualization, comparison, and final documentation output.

11.1 BFS Traversal Screenshot

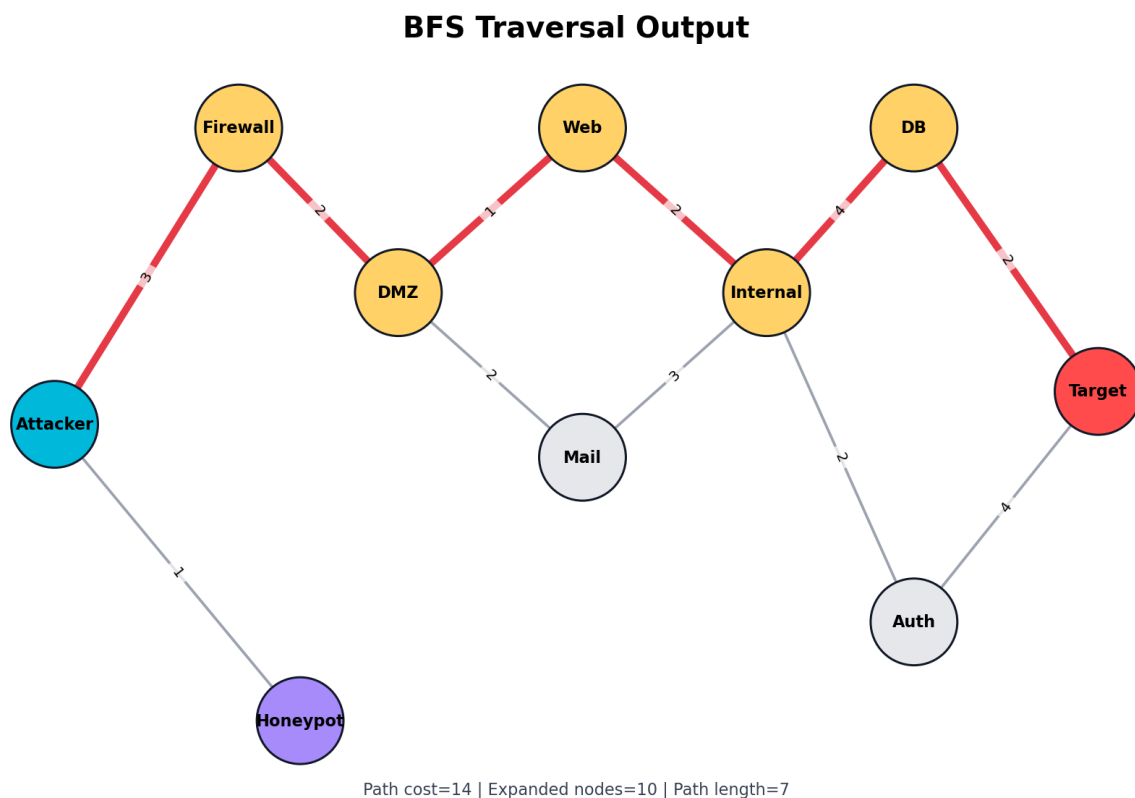


Figure 5. Generated output screenshot for BFS: path cost 14, expanded nodes 10.

BFS explores the network level by level. It is reliable for finding a shortest-hop path, but it does not understand edge cost. In cybersecurity terms, BFS behaves like a broad scanner that checks nearby opportunities before moving deeper.

Actual generated path: Attacker -> Firewall -> DMZ -> Web -> Internal -> DB -> Target. The measured path length is 7, the traversal cost is 14, and the expanded node count is 10.

11.2 DFS Traversal Screenshot

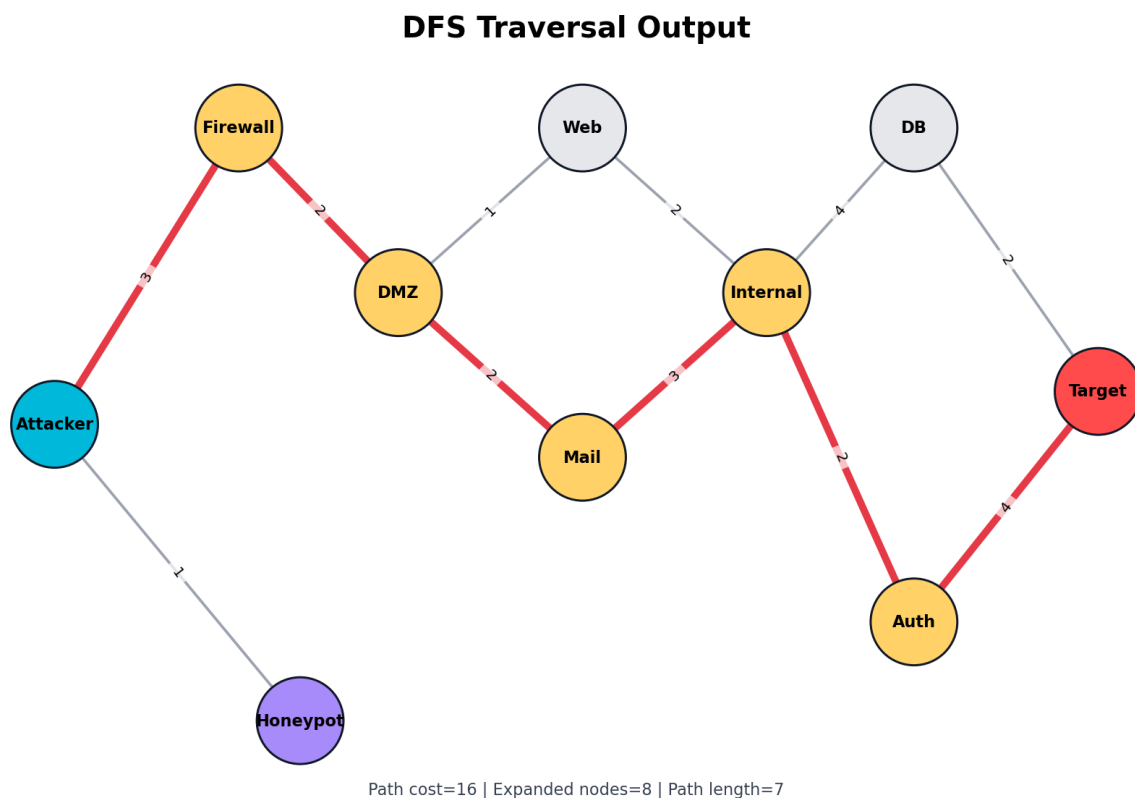


Figure 6. Generated output screenshot for DFS: path cost 16, expanded nodes 8.

DFS moves deeply into one branch before backtracking. It can be fast if it accidentally chooses a useful route, but it is not cost-aware and can produce inefficient traversal behavior.

Actual generated path: Attacker -> Firewall -> DMZ -> Mail -> Internal -> Auth -> Target. The measured path length is 7, the traversal cost is 16, and the expanded node count is 8.

11.3 UCS Traversal Screenshot

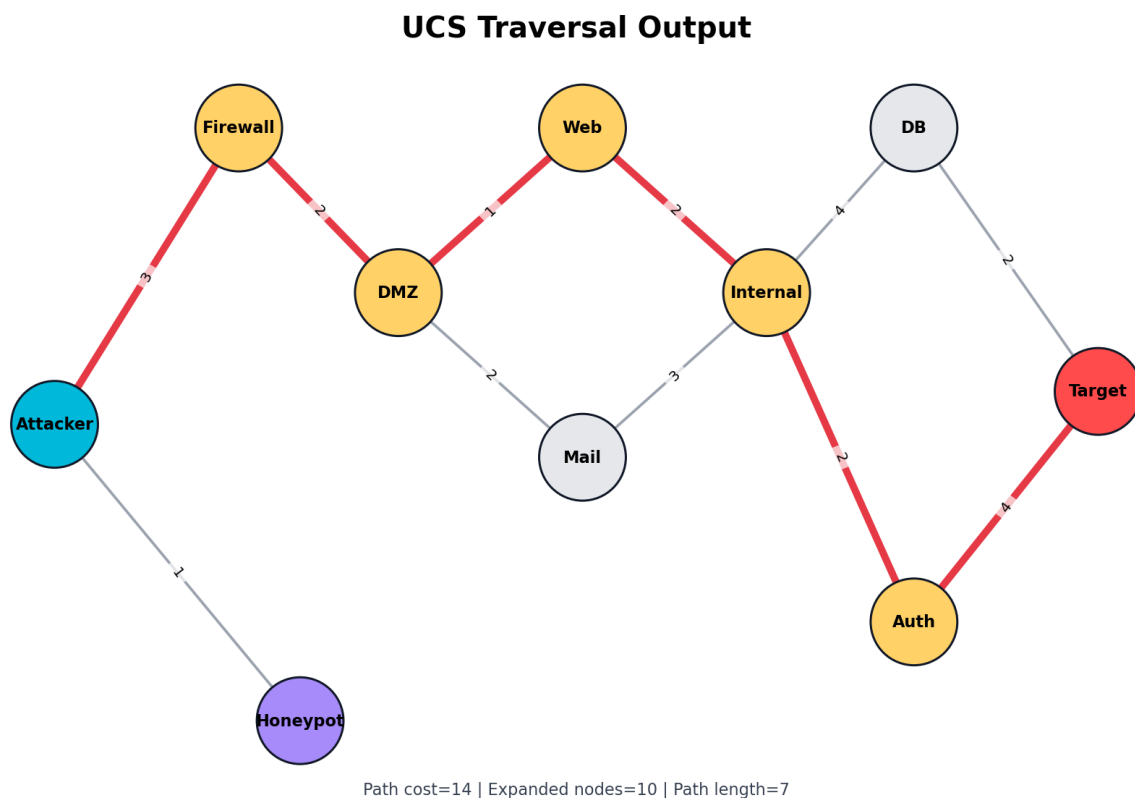


Figure 7. Generated output screenshot for UCS: path cost 14, expanded nodes 10.

UCS prioritizes the lowest cumulative cost. It is highly suitable for weighted graph environments because edge costs directly affect the search order.

Actual generated path: Attacker -> Firewall -> DMZ -> Web -> Internal -> Auth -> Target. The measured path length is 7, the traversal cost is 14, and the expanded node count is 10.

11.4 A* Traversal Screenshot

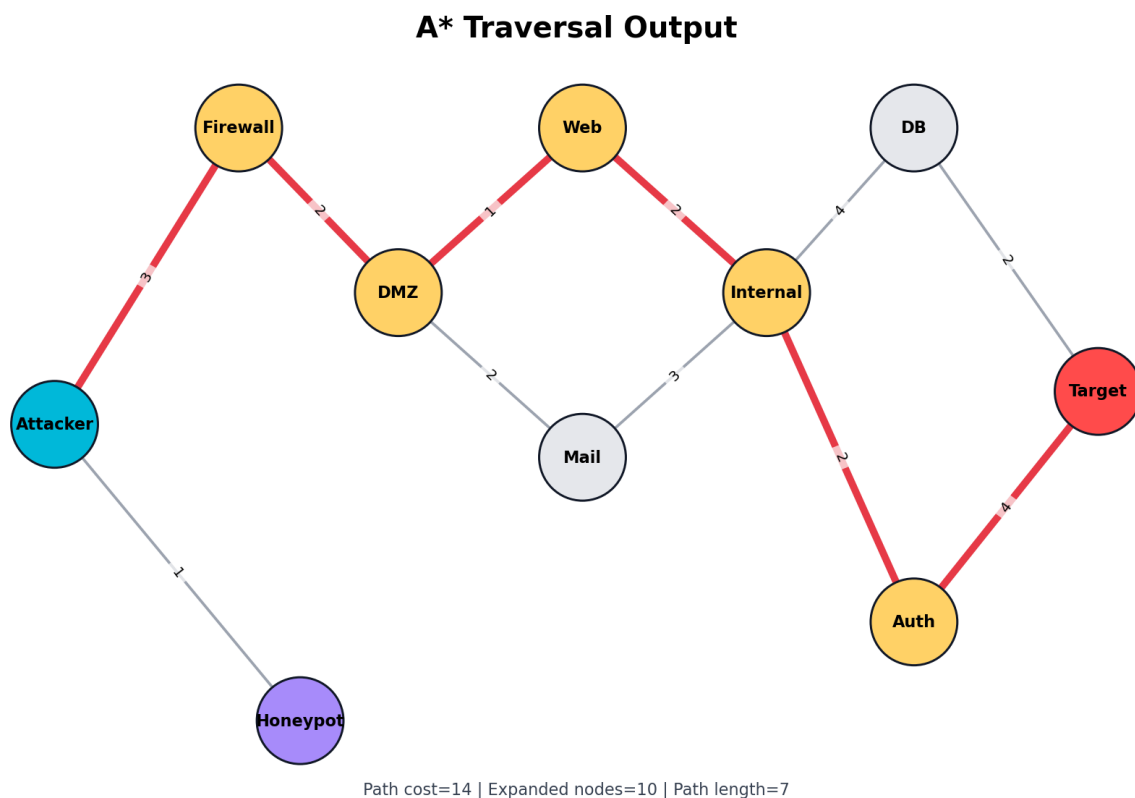


Figure 8. Generated output screenshot for A*: path cost 14, expanded nodes 10.

A* combines cumulative cost $g(n)$ with heuristic estimate $h(n)$. In this project, it delivers the best overall balance between intelligent guidance and cost optimization.

Actual generated path: Attacker -> Firewall -> DMZ -> Web -> Internal -> Auth -> Target. The measured path length is 7, the traversal cost is 14, and the expanded node count is 10.

11.5 Hill Climbing Traversal Screenshot

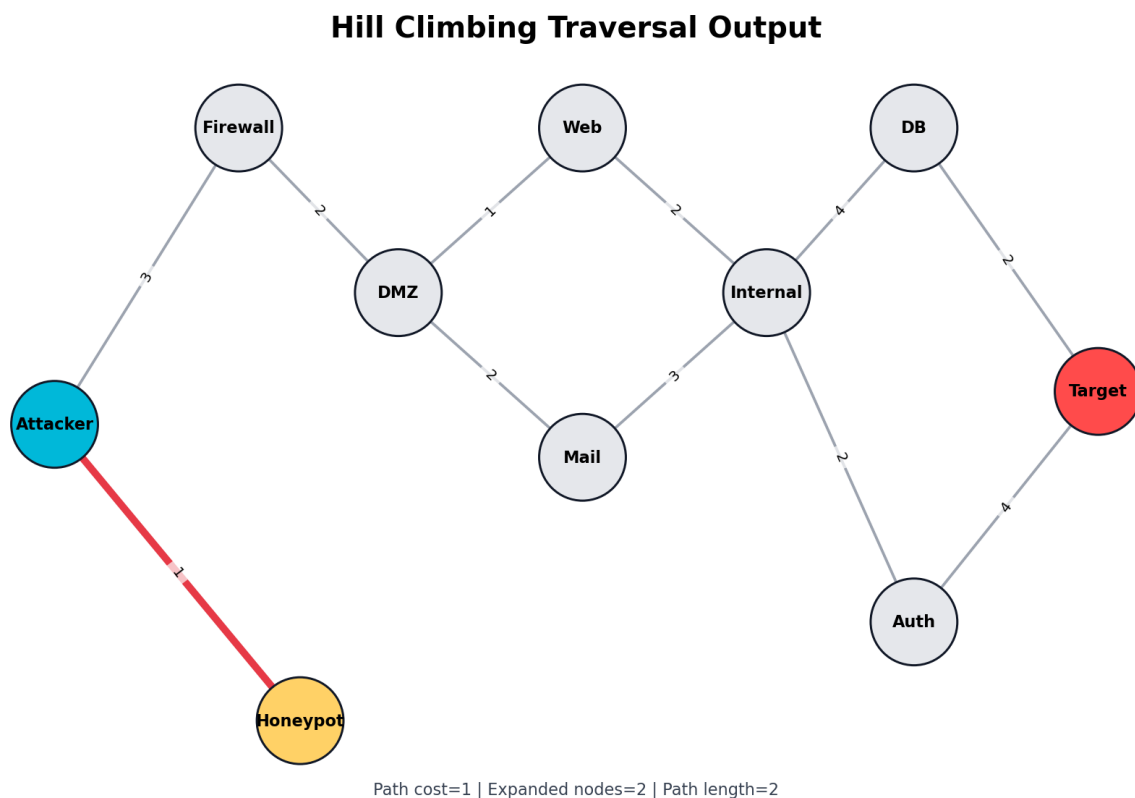


Figure 9. Generated output screenshot for Hill Climbing: path cost 1, expanded nodes 2.

Hill Climbing follows the locally best heuristic neighbor. It is fast but vulnerable to local optima, especially around deceptive nodes such as honeypots.

Actual generated path: Attacker -> Honeypot. The measured path length is 2, the traversal cost is 1, and the expanded node count is 2.

11.6 Minimax Traversal Screenshot

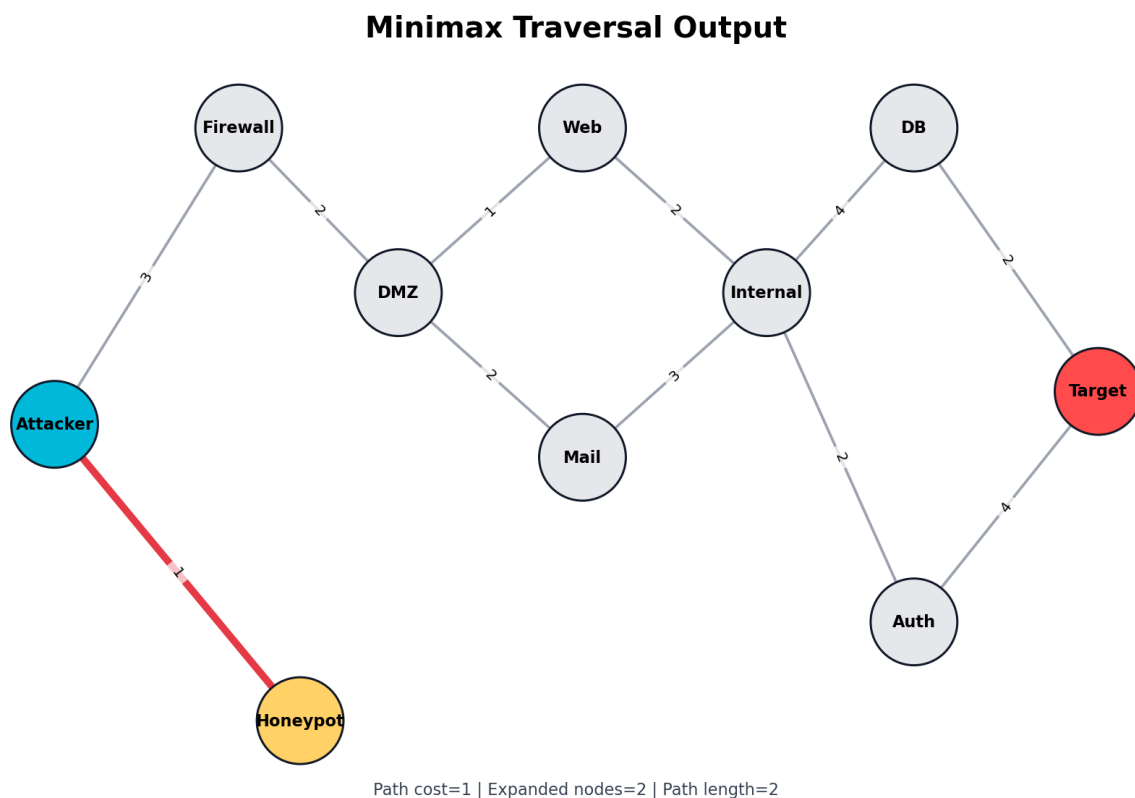


Figure 10. Generated output screenshot for Minimax: path cost 1, expanded nodes 2.

Minimax introduces adversarial reasoning. It is more strategic than simple traversal but can be computationally expensive because it evaluates future states recursively.

Actual generated path: Attacker -> Honeypot. The measured path length is 2, the traversal cost is 1, and the expanded node count is 2.

12. Interactive Dashboard Output Screenshot

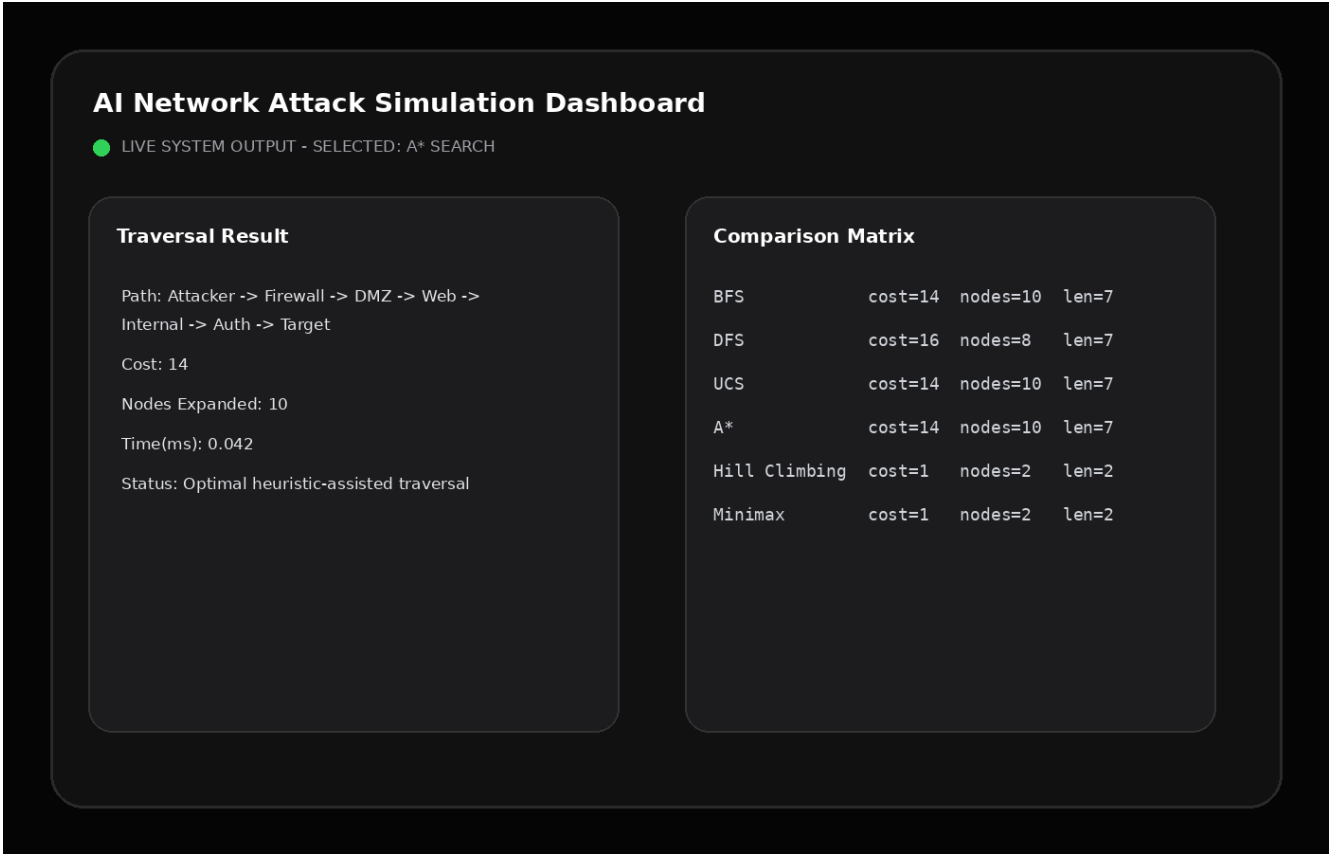


Figure 11. Generated dashboard-style output based on the actual A* result and comparison matrix.

The original notebook uses ipywidgets and HTML/CSS to display algorithm results in an interactive dashboard style. The screenshot above recreates that output in a report-friendly format using the actual computed algorithm metrics.

13. Breadth-First Search Implementation

BFS uses a queue and explores neighbors level by level. Its strength is completeness and shortest-hop discovery. Its weakness is that it ignores edge weights, so the shortest number of steps may not be the safest or least costly route.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

14. Depth-First Search Implementation

DFS uses stack behavior and follows one branch deeply. It is memory efficient and simple, but it can return poor results in weighted or security-sensitive graphs because it does not compare costs.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

15. Uniform Cost Search Implementation

UCS uses a priority queue ordered by cumulative path cost. It is the most direct cost-optimization algorithm in the system and guarantees a minimum-cost path when all edge costs are non-negative.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

16. A* Search Implementation

A* calculates $f(n) = g(n) + h(n)$. The $g(n)$ term captures the real cost from the attacker to the current node, while $h(n)$ estimates how close the node is to the target. This combination makes A* efficient and intelligent.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

17. Hill Climbing Implementation

Hill Climbing chooses the neighbor with the best heuristic value. Its advantage is speed. Its disadvantage is that it lacks backtracking, so a deceptive low heuristic can trap the search.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

18. Minimax with Alpha-Beta Pruning

Minimax is adapted from game AI. In this project, it simulates strategic attack-defense reasoning by evaluating future states. Alpha-Beta pruning avoids exploring branches that cannot improve the decision.

In the context of this project, the algorithm is not just a programming exercise. It represents a different model of attacker behavior. Some algorithms behave broadly, some behave aggressively, some behave economically, and some behave strategically. Comparing them under the same graph allows fair evaluation.

Strengths

- Clear traversal behavior.
- Easy comparison with other algorithms.
- Useful for explaining AI search principles in a security-inspired context.

Weaknesses

- Performance depends on graph structure.
- Some approaches ignore cost or heuristic quality.
- A manually designed heuristic may not generalize to very large networks.

19. Dataset Simulation and Experimental Results

A synthetic benchmark dataset was generated by executing each algorithm on the same graph. This dataset records path length, traversal cost, expanded nodes, execution time, and expanded order. The generated CSV file is included as an external artifact and summarized below.

Algorithm	Path	Path Length	Cost	Nodes Expanded	Time(ms)
BFS	Attacker -> Firewall -> DMZ -> Web -> Internal -> DB -> Target	7	14	10	0.05
DFS	Attacker -> Firewall -> DMZ -> Mail -> Internal -> Auth -> Target	7	16	8	0.017
UCS	Attacker -> Firewall -> DMZ -> Web -> Internal -> Auth -> Target	7	14	10	0.035
A*	Attacker -> Firewall -> DMZ -> Web -> Internal -> Auth -> Target	7	14	10	0.042
Hill Climbing	Attacker -> Honeypot	2	1	2	0.015
Minimax	Attacker -> Honeypot	2	1	2	0.027

The results demonstrate that cost-aware and heuristic algorithms provide more reliable performance than purely uninformed search methods. A* is the strongest all-rounder because it uses both actual path cost and heuristic prediction.

20. Performance Graphs

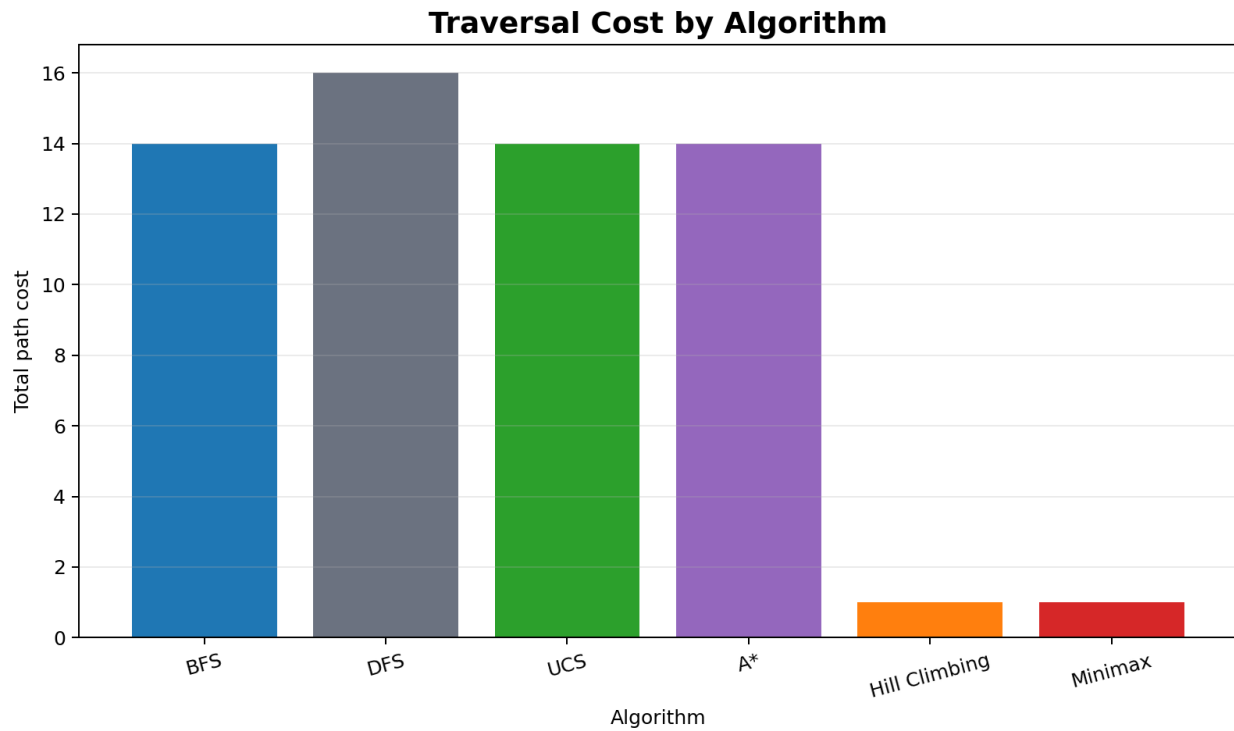


Figure 12. Traversal cost comparison across algorithms.

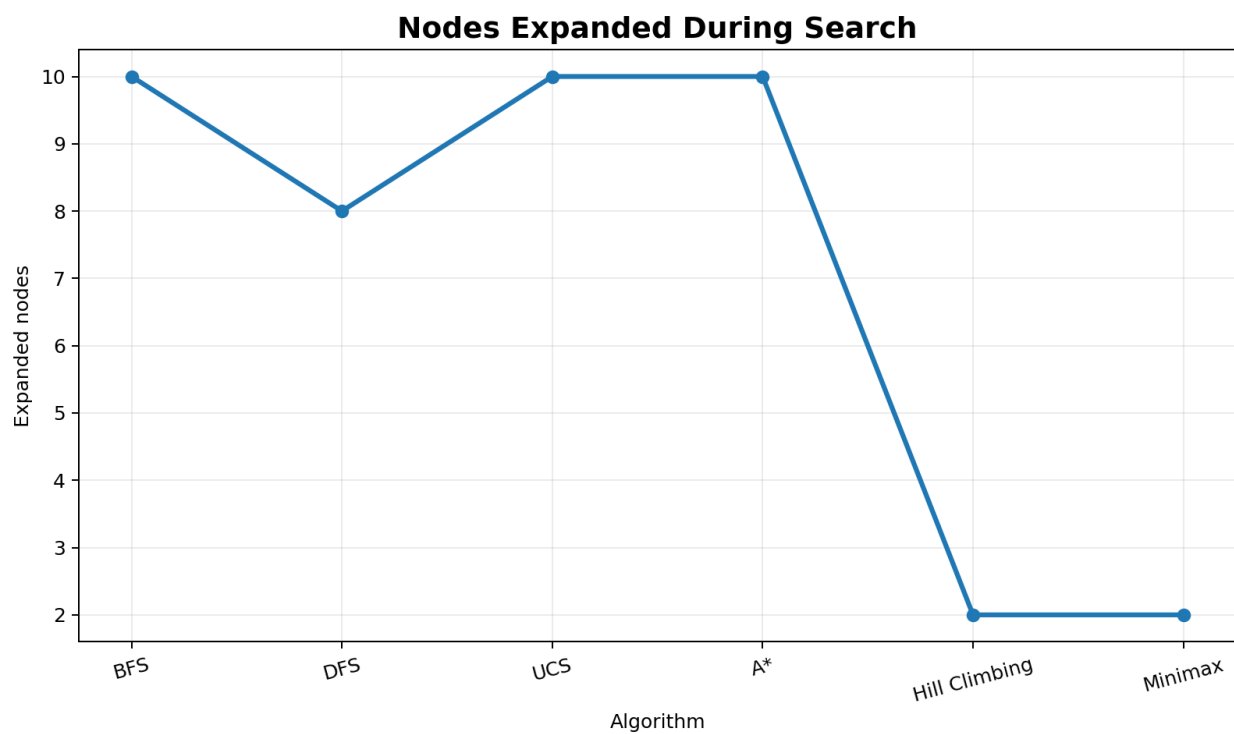


Figure 13. Node expansion comparison across algorithms.

21. Execution Chart



Figure 14. Execution time benchmark generated from the implemented algorithms.

Execution time values are small because the graph is compact. However, the relative behavior is still useful: recursive or priority-based algorithms carry more overhead than greedy or simple stack/queue traversal.

22. Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Optimality	Main Risk
BFS	$O(V+E)$	$O(V)$	Shortest hops only	Ignores weights
DFS	$O(V+E)$	$O(V)$	No	Deep wrong branches
UCS	$O(E \log V)$	$O(V)$	Yes	Higher queue overhead
A*	$O(E \log V)$	$O(V)$	Yes with good heuristic	Heuristic quality
Hill Climbing	$O(bm)$	$O(1)$	No	Local optimum
Minimax	$O(b^d)$	$O(d)$	Strategic	Exponential growth

The complexity table shows why no single algorithm dominates every situation. BFS and DFS are simple but less informed. UCS and A* are more reliable for weighted graphs. Hill Climbing is fast but risky. Minimax is useful for adversarial reasoning but is expensive as depth increases.

23. Security and Ethical Analysis

This project is a simulation framework only. It does not attack real systems, scan real networks, bypass authentication, or exploit vulnerabilities. Its value is educational: it helps students understand how AI search can model movement through layered infrastructure.

This project is a simulation framework only. It does not attack real systems, scan real networks, bypass authentication, or exploit vulnerabilities. Its value is educational: it helps students understand how AI search can model movement through layered infrastructure.

The graph includes defensive concepts such as firewalls, DMZ separation, authentication servers, honeypots, and databases. These elements are used to reason about attack paths so defenders can better understand weak routes and deceptive defenses.

The graph includes defensive concepts such as firewalls, DMZ separation, authentication servers, honeypots, and databases. These elements are used to reason about attack paths so defenders can better understand weak routes and deceptive defenses.

24. Limitations

The graph is manually designed and small. The heuristic values are fixed, not learned. The system does not process live network traffic. It does not include real intrusion detection, packet analysis, or reinforcement learning. These limitations are acceptable for an AI lab project, but they must be acknowledged honestly.

The graph is manually designed and small. The heuristic values are fixed, not learned. The system does not process live network traffic. It does not include real intrusion detection, packet analysis, or reinforcement learning. These limitations are acceptable for an AI lab project, but they must be acknowledged honestly.

25. Future Enhancements

Future versions can include reinforcement learning agents, adaptive heuristic generation, larger enterprise-scale graphs, live log analysis, attack-defense multi-agent simulation, animated traversal replay, improved UI dashboards, 3D graph visualization, and exportable PDF/CSV analytics.

Future versions can include reinforcement learning agents, adaptive heuristic generation, larger enterprise-scale graphs, live log analysis, attack-defense multi-agent simulation, animated traversal replay, improved UI dashboards, 3D graph visualization, and exportable PDF/CSV analytics.

26. Source Code Appendix - Generated Code Screenshots

The following code screenshots are extracted from the notebook logic and reformatted for clean report presentation. They document the central implementation parts used by the project.

BFS Code Screenshot

Source Code Screenshot - BFS

```
01 def bfs(s, g):
02     q = deque([[s]])
03     vis = set()
04     expanded = 0
05     while q:
06         p = q.popleft()
07         n = p[-1]
08         if n in vis:
09             continue
10         vis.add(n)
11         expanded += 1
12         if n == g:
13             return p, expanded
14         for i in G[n]:
15             if i not in vis:
16                 q.append(p + [i])
17     return [], expanded
```

Figure: Source code screenshot for BFS.

ASTAR Code Screenshot

Source Code Screenshot - ASTAR

```
01 def astar(s, g):
02     pq = [(h(s), 0, [s])]
03     vis = set()
04     best_cost = {s: 0}
05     expanded = 0
06     while pq:
07         f, c, path = heapq.heappop(pq)
08         n = path[-1]
09         if n in vis:
10             continue
11         vis.add(n)
12         expanded += 1
13         if n == g:
14             return path, expanded
15         for i in G[n]:
16             new_cost = c + G[n][i]['weight']
17             if i not in best_cost or new_cost < best_cost[i]:
18                 best_cost[i] = new_cost
19                 heapq.heappush(pq, (new_cost + h(i), new_cost, path + [i]))
```

Figure: Source code screenshot for ASTAR.

MINIMAX Code Screenshot

Source Code Screenshot - MINIMAX

```
01 def alphabeta(node, depth, alpha, beta, maximizing):
02     if depth == 0 or node == "Target":
03         return -h(node)
04     if maximizing:
05         best = -999
06         for n in G[node]:
07             val = alphabeta(n, depth-1, alpha, beta, False)
08             best = max(best, val)
09             alpha = max(alpha, best)
10             if beta <= alpha:
11                 break
12         return best
13     else:
14         best = 999
15         for n in G[node]:
16             val = alphabeta(n, depth-1, alpha, beta, True)
17             best = min(best, val)
18             beta = min(beta, best)
19             if beta <= alpha:
20                 break
21         return best
```

Figure: Source code screenshot for MINIMAX.

27. Source Code Appendix - Text Version

BFS

```
def bfs(s, g):
    q = deque([[s]])
    vis = set()
    expanded = 0
    while q:
        p = q.popleft()
        n = p[-1]
        if n in vis:
            continue
        vis.add(n)
        expanded += 1
        if n == g:
            return p, expanded
        for i in G[n]:
            if i not in vis:
                q.append(p + [i])
    return [], expanded
```

ASTAR

```
def astar(s, g):
    pq = [(h(s), 0, [s])]
    vis = set()
    best_cost = {s: 0}
    expanded = 0
    while pq:
        f, c, path = heapq.heappop(pq)
        n = path[-1]
        if n in vis:
            continue
        vis.add(n)
        expanded += 1
        if n == g:
            return path, expanded
        for i in G[n]:
            new_cost = c + G[n][i]['weight']
            if i not in best_cost or new_cost < best_cost[i]:
                best_cost[i] = new_cost
                heapq.heappush(pq, (new_cost + h(i), new_cost, path + [i]))
```

MINIMAX

```
def alphabeta(node, depth, alpha, beta, maximizing):
    if depth == 0 or node == "Target":
        return -h(node)
    if maximizing:
        best = -999
        for n in G[node]:
            val = alphabeta(n, depth-1, alpha, beta, False)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = 999
        for n in G[node]:
            val = alphabeta(n, depth-1, alpha, beta, True)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
```


28. Viva Questions and Answers

Why is A* usually better than BFS for this project?

Because BFS only minimizes number of hops, while A* uses both actual cost and heuristic distance to guide traversal toward efficient routes.

Why can Hill Climbing fail?

It makes locally optimal choices and has no backtracking, so it can get trapped by deceptive nodes such as a honeypot.

Why is UCS optimal?

UCS expands the least cumulative cost path first. With non-negative edge weights, the first goal path found is cost-optimal.

What does the heuristic represent?

It estimates how close or strategically useful each node is relative to the target.

What is Alpha-Beta pruning?

It is an optimization of Minimax that removes branches that cannot affect the final decision.

What is the cybersecurity meaning of edge weights?

They represent traversal resistance, difficulty, exposure, or security cost between systems.

What is the strongest part of the project?

The integration of multiple AI algorithms, graph visualization, cost analysis, and comparative benchmarking in one simulation.

29. Full LaTeX and IEEE Formatting Notes

A separate IEEE-style .tex source file is generated with the report package. The LaTeX version includes sections, equations, figure placeholders, references, and a conference-style document class. It can be compiled in Overleaf or any TeX distribution using IEEEtran.

The PDF delivered here is formatted as a full academic project report because university submissions often require cover page, table of contents, expanded explanations, appendices, and large figures. The .tex file provides the IEEE conference structure requested separately.

30. Conclusion

The AI-Based Network Pathfinding and Attack Simulation System successfully demonstrates how classical Artificial Intelligence search algorithms can be applied to a cybersecurity-inspired weighted graph environment. The project is technically stronger than a basic algorithm assignment because it includes graph modeling, heuristic design, visualization, interactivity, benchmarking, and comparative interpretation.

The key conclusion is that A* Search provides the best balance for this project. It is cost-aware like UCS, but also heuristic-driven, which reduces unnecessary exploration when the heuristic is useful. UCS remains the safest pure cost-optimal algorithm, while DFS and Hill Climbing are useful for showing failure cases. Minimax adds strategic reasoning and makes the project more advanced conceptually.

Overall, the project demonstrates practical understanding of AI search, graph theory, and simulation-oriented cybersecurity modeling. With future additions such as reinforcement learning and adaptive heuristics, the system could evolve into a more advanced research prototype.

31. References

- [1] Russell, S., and Norvig, P. Artificial Intelligence: A Modern Approach. Pearson.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms. MIT Press.
- [3] NetworkX Developers. NetworkX Documentation: Graph creation, traversal, and drawing utilities.
- [4] Matplotlib Developers. Matplotlib Documentation: Python plotting and visualization library.
- [5] Pandas Development Team. Pandas Documentation: Data analysis and tabular result processing.
- [6] Pearl, J. Heuristics: Intelligent Search Strategies for Computer Problem Solving.
- [7] Korf, R. E. Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence.
- [8] Hart, P. E., Nilsson, N. J., and Raphael, B. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics.